UNIVERSITY OF PENNSYLVANIA ESE 5700

Project 1, Part 3: the MAC unit

Tyrone Marhguy • Victor Wanjohi

Cadence GPDK 18 nm $V_{DD} = 0.8$ V, TT, 27°C schematic level, $C_{BL} = 3.5$ fF Tests A, B, C pass 17 cycles START→DONE area ≈ 290 μm^2 (est.) $T_{CLK,\min}$ = **[TODO:]**

1. Team and architecture

Member	Contribution
Tyrone Marhguy	**[TODO: fill in]**
Victor Wanjohi	**[TODO: fill in]**

Two Part 2 macros hold W and X; a static-CMOS array multiplier, a 12-bit ripple accumulator, a 4-bit counter and a two-state-bit controller (all registers are transmission-gate master–slave flip-flops) compute ACC = $\sum_{i=0}^{15} W[i]X[i]$. The read is pipelined against the multiply-accumulate: the macro's output latch is the only pipeline stage, so a dot product takes 17 cycles.

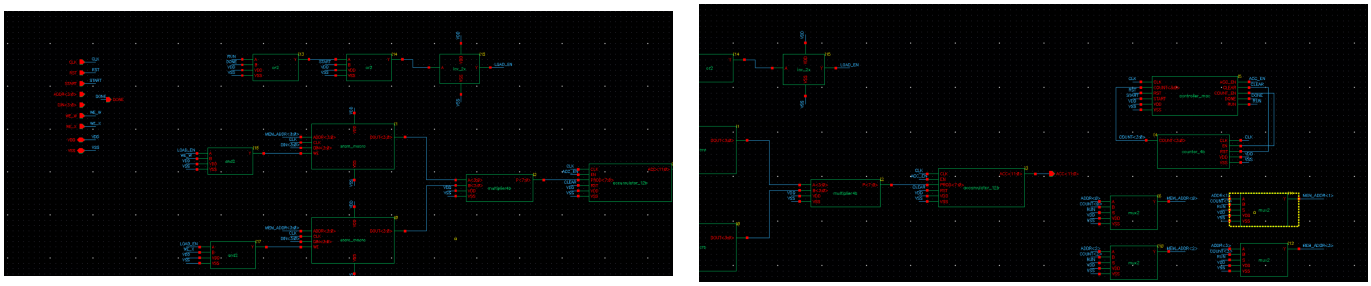


Figure 1: `top`. Left: macros (WE = LOAD_EN·WE_W / WE_X), multiplier, accumulator, LOAD_EN = $\overline{\text{RUN} + \text{DONE} + \text{START}}$. Right: controller, counter, address mux (COUNT while RUN).

2. Register (3.1)

Positive-edge master–slave flip-flop from two static TG latches (master transparent while CLK = 0, slave while CLK = 1); `tg` 2+2 fins, `inv_2x` 8/4 fins, 100 fins. Chosen because it is fully static (holds with the clock stopped), needs one clock, and reuses verified cells; the cost is two local clock inverters (clock load). `register_1b` adds a `mux2` enable and an `and2` synchronous reset (RST active-high), 152 fins.

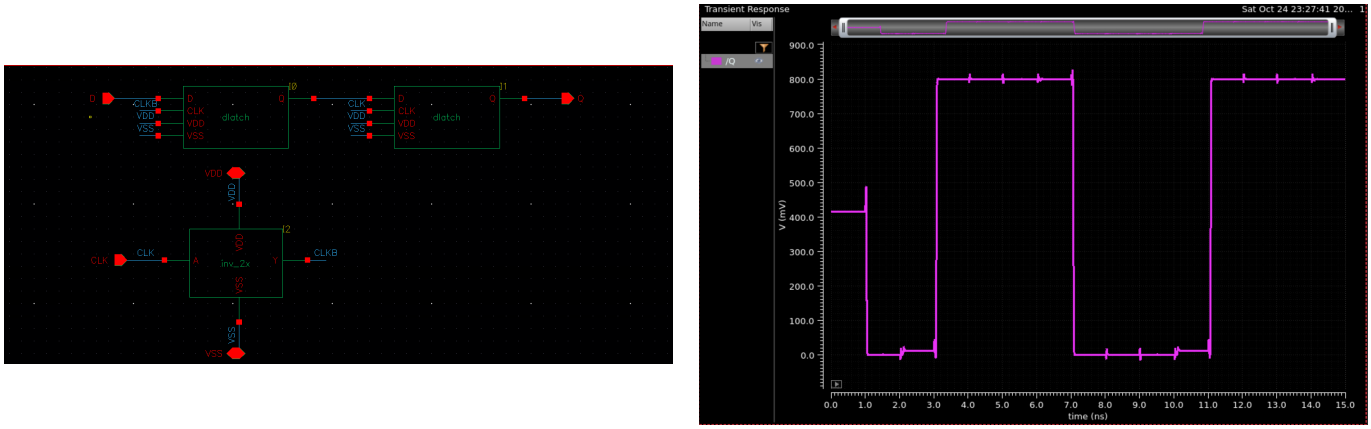


Figure 2: `d_ff` (left) and `tb_dff`: Q captures on each rising CLK, $t_{cq} \approx 100$ ps (CLK 50% → Q 50%, $C_Q = 1$ fF).

Quantity	Value	Simulation
t_{cq} (rise/fall)	100 ps	`tb_dff`, D stable 1 ns before the edge
t_{su} / t_h	[TODO:] / [TODO:]	D-to-CLK offset sweep, 10% t_{cq} push-out
C_{CLK}	[TODO:] fF	CLK-source charge per cycle $/V_{DD}$
Energy per clock	[TODO:] fJ	$\int V_{DD} I_{DD}\, dt$ over one cycle

3. Datapath (3.2)

Multiplier: 16 `and2` partial products and three 4-bit ripple rows (12 full adders, 2096 fins). **Accumulator:** 12-bit ripple adder ($B = \{0000, P\}$, $3600 < 4096$) into 12 `register_1b` (3600 fins). Full adder: 2 `xor2` (static, input inverters; NMOS 2/4, PMOS 2 fins), 2 `and2`, 1 `or2`, 148 fins.

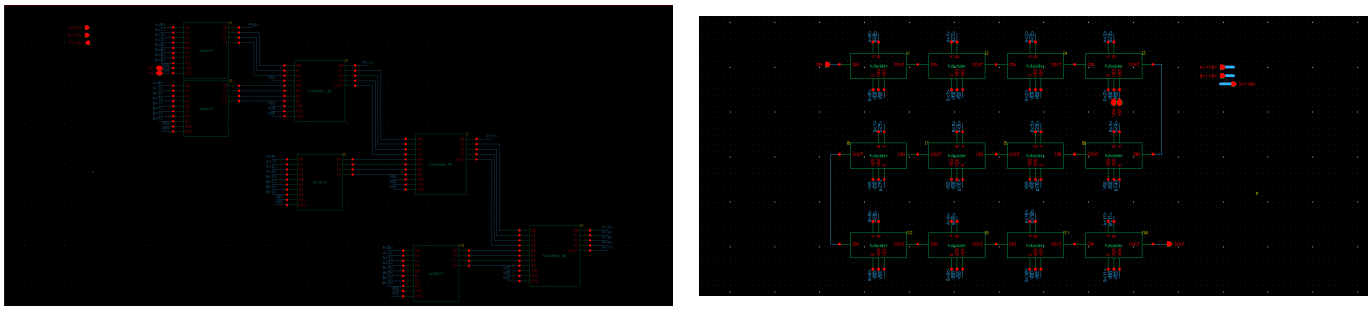


Figure 3: `multiplier4b` (row j ANDs A with B_j) and `fulladder_12b`.

Logical effort ($\tau = 10.5$ ps from Part 2 FO4 = 53 ps; f from fin ratios to `inv_u`, worst edge):

Full-adder arc	d	Path	Delay
$C_{in} \to C_{out}$	17τ	multiplier: and2, $3\times(A/B \to C_{out}, C_{in} \to S)$, 3 carries to P_7	$176\tau = 1.85$ ns
$C_{in} \to S$	13τ	accumulator: P_4, carries to bit 11	$145\tau = 1.5$ ns
$A/B \to S$	26τ	register input (`mux2`+`and2`)	0.1 ns
$A/B \to C_{out}$	30τ		

The `xor2` core dominates: its 2-fin series PMOS stack is $\approx 4\times$ weaker than `inv_u` (12 of the 26τ of a sum arc). Upsizing it to 4-fin stacks would cut the multiplier to ≈ 1.3 ns (LE).

Exhaustive multiplier test: all 256 pairs; the check samples $|P - A \cdot B|$ at $0.95\, t_v$ in every vector (spec < 0.5).

t_v (ns)	1.0	1.1	1.2	**1.3**	1.4	1.5
errmax	44	14	0.62	**0.17**	0.14	0.12
result	fail	fail	fail	**pass**	pass	pass

256/256 correct; **measured worst case 1.2 ns** (1.14–1.24 ns; vector $B = 15$, A $3 \to 4$) against **1.85 ns predicted**: LE charges every stage its slow edge and the full load.

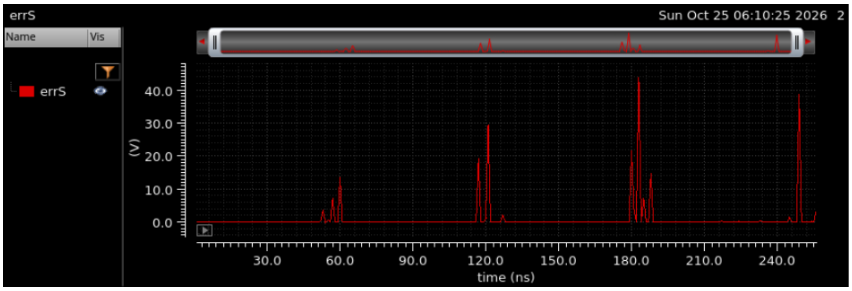
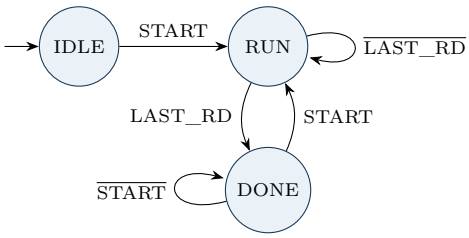


Figure 4: Sampled error at $t_v = 1$ ns: zero except a few carry-heavy vectors still settling at 0.95 ns.

4. Sequencer (3.3)



State (S_0, DONE); RUN $= S_0\overline{\text{DONE}}$. START (not in RUN) raises CLEAR (ACC, counter) and sets S_0. In RUN: COUNT_EN $= \overline{\text{LAST}}$ (count 0–15, hold at 15), ACC_EN = VALID (RUN one cycle late: pipeline). LAST_READ, set the cycle after count = 15, ends RUN and sets DONE, which holds until START. RST clears all. Counter: synchronous toggle chain, 4 `d_ff`.

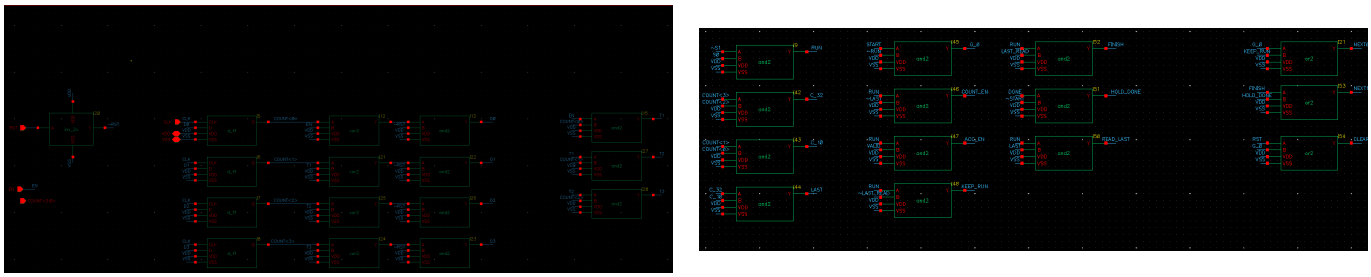


Figure 5: `counter_4b` and `controller_mac` next-state logic (state registers in the long report).

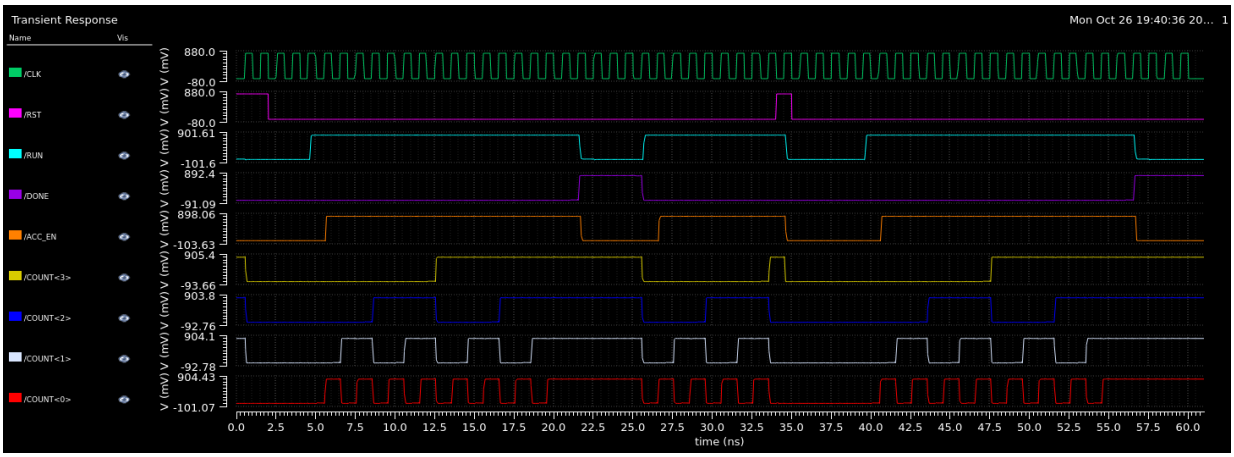


Figure 6: `controller_mac`+`counter_4b`, vector-file bench (60 edges, **60/60 checks pass**; counter alone 38/38): START at edge 5, DONE after edge 22 (**17 edges**), restart from DONE, START ignored in RUN, RST abort, full second run.

One dot product (Test A, $T = 10$ ns): START edge 332 ns → RUN, count 0; edge n reads address $n-1$ and DOUT is valid ≈ 0.5 ns later; ACC_EN from 342 ns, first accumulate at 352 ns; 16th at 502 ns with DONE (Fig. 7). **[TODO: Drawn timing diagram.]**

5. Timing closure (3.4)

Critical path (one stage after the macro's output latch): CLK → SRAM access → multiplier → adder → register setup.

Term	Value	Source
t_{SRAM}	0.49 ns	Part 2 worst-case access
t_{mult}	1.85 ns LE / 1.2 ns measured	§3
t_{add}+ register input	1.6 ns LE	§3
t_{su}	**[TODO:]**	§2
Hand $T_{\min}$	**4.0 ns** $+ t_{su}$ (3.3 ns with the measured multiplier)	sum
Measured $T_{\min}$	**[TODO:]**	shortest T at which A, B, C pass

Hold: counter → `mux2` → macro address latch, which closes at the same edge; requires $t_{cq,\min} + t_{\mathrm{mux}} > t_h$. **[TODO: margin]**

6. Verification (3.5)

Vector-file stimulus: reset, load W then X (one word per cycle, inputs change on the falling edge), START, run, $T = 10$ ns.

Test	W, X	Expected	ACC	Result
A	handout vectors	845 (0x34D)	0x34D	**pass**
B	all 15	3600 (0xE10)	0xE10	**pass**
C	5/10 alternating	800 (0x320)	0x320	**pass**

DONE rises 17 edges after START in all three. **Bug found here:** ACC first read 0 because the macro's output-latch enable was a 150 ps pulse that closed before the read inverter switched; it is now level-sensitive on CLK high (RD_EN = CLKBUF · $\overline{\mathrm{WE_L}}$).

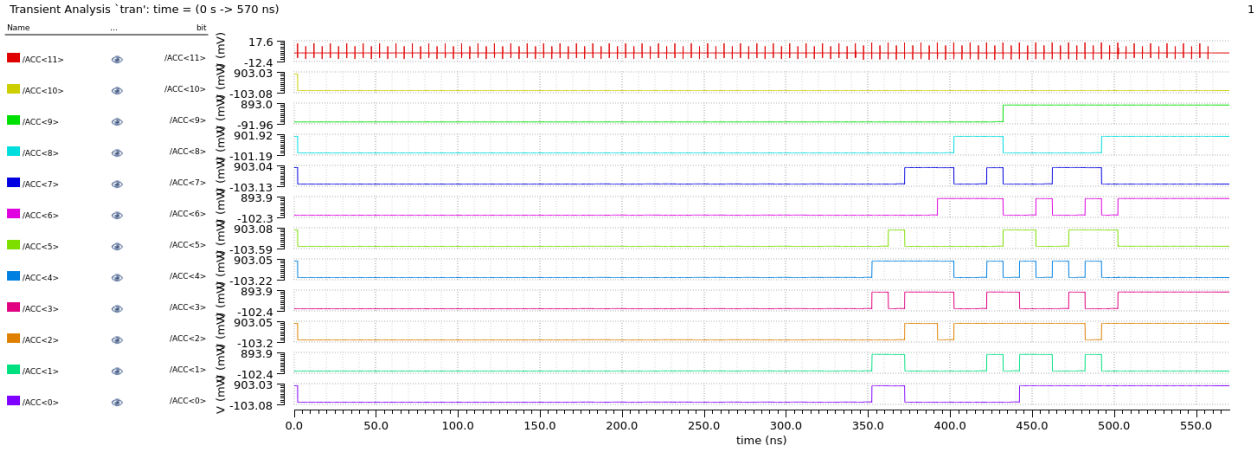


Figure 7: Test A: ACC steps 27, 51, 156, . . . , 805, **845 (0x34D)** at 502 ns and holds. **[TODO: add DONE trace]**

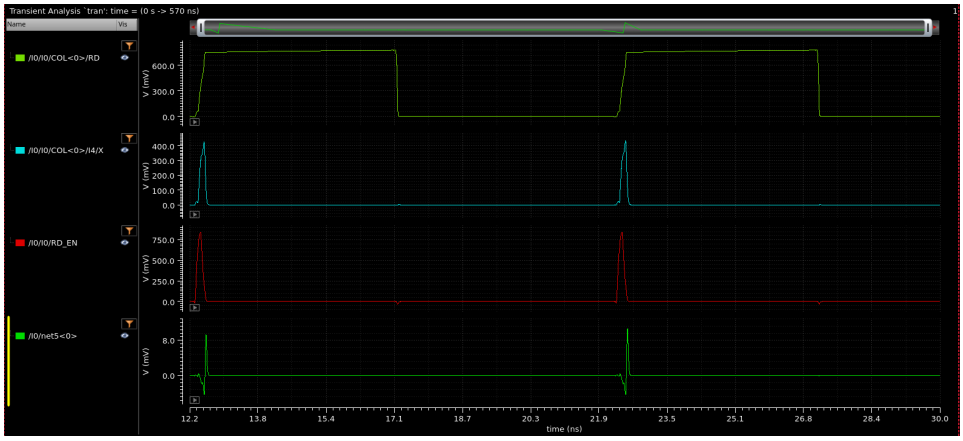


Figure 8: Before the fix: RD rises after the read, but the RD_EN pulse closes first; the latch node X peaks near 0.4 V and returns to 0.

7. Metrics

Metric	Value
$T_{CLK,\min}$; N_{cycles}; t_{dot}	**[TODO:]**; 17; **[TODO:]**
Area (2 macros + datapath 5696 fins + sequencer 1848 fins, $D = 28$)	$109 + 138 + 45 \approx 290$ μm^2 (est.)
E_{dot} (Test A/B/C); energy per MAC with / without memory	**[TODO:]**
$\text{FOM}_{MAC} = \text{Area} \cdot E_{dot} \cdot t_{dot}$	**[TODO:]**

8. What limited the design

The multiply-accumulate path, not the memory, sets the clock: the ripple multiplier alone takes 1.2 ns, over twice the 0.49 ns SRAM read, before the ripple adder and setup. Both use one small full adder whose 2-fin `xor2` stacks dominate the delay; the datapath is also the largest area (138 μm^2, more than both macros). Next steps: carry-save multiplier, carry-select final adder, upsized `xor2`, and a register after the multiplier (18 cycles, within the 20-cycle limit). **[TODO: energy]**